

Team Details :

1. Name: Ahmed Saad Ali ID: 2401249800
2. Name: Adham Ahmed Abdelmawla ID: 2401243185
3. Name: Badr Ramadan Saad ID: 2401242155
4. Name: Mohamed Saad Tantawy ID: 2401248635
5. Name: Yahia Abdalhalim Awad Abdalhalim ID:2401247051
6. Name: Mahmoud Taqi Eldin Ibrahim ID:2401242945

1. System Overview

This Java-based university management system provides a robust framework for handling student enrollments, course administration, and academic operations using efficient data structures. The implementation focuses on clean object-oriented design with clear separation of concerns.

2. Core Class Structure

2.1 Student Class

```
public class Student {
    private courseNode courseHead; // Head of enrolled courses linked list
    private courseNode courseTail; // Tail pointer for O(1) appends
    int studentID;                // Unique identifier
    Student next;                 // Next student in university list
    int counterOfCourse;          // Tracks enrolled course count

    // Constructors initialize with ID and optional next student
    public Student(int studentID) { /*...*/ }
    public Student(int studentID, Student n) { /*...*/ }

    // Core functionality methods
    public boolean isEmpty() { /* Checks if no courses enrolled */ }
    public void addcourse(int courseID) { /* Adds course with capacity check */ }
    public void removeCourse(int courseID) { /* Removes specific course */ }
    public void printLists() { /* Displays enrolled courses */ }
    public void sortCourseList() { /* Bubble sort implementation */ }
    public boolean isNormal() { /* Validates 2-7 course range */ }
    public boolean isStudentInTheCourse(int courseID) { /* Membership check */ }
}
```

2.2 Course Class

```
public class Course {
```

```

studentNode studentHead;    // Head of enrolled students list
studentNode studentTail;    // Tail pointer for efficient additions
int courseID;               // Unique course identifier
Course next;                // Next course in university list
int counterOfStudent = 0;    // Tracks current enrollment

// Constructors initialize course with ID
public Course(int courseID) { /*...*/ }
public Course(int courseID, Course n) { /*...*/ }

// Core functionality methods
public boolean isEmpty() { /* Checks if no students enrolled */ }
public void addStudent(int studentID) { /* Adds student with capacity check */ }
public void removeStudent(int studentID) { /* Removes specific student */ }
public void printLists() { /* Displays enrolled students */ }
public boolean isFull() { /* Checks 30 student capacity */ }
public void sortStudentList() { /* Bubble sort implementation */ }
public boolean isCourseInStudentList(int studentID) { /* Membership check */ }
}

```

3. University Management Core

3.1 University Class

```

public class University {
    private Student studentHead, studentTail; // Student linked list
    private Course courseHead, courseTail;    // Course linked list
    private Stack undo, redo;                 // Operation history stacks

    // Core management methods
    public void addStudent(int studentID) { /* Adds student to university */ }
    public void addCourse(int courseID) { /* Adds course to university */ }
    public void removeStudent(int studentID) { /* Removes student from all records */ }
    public void removeCourse(int courseID) { /* Removes course from all records */ }

    // Enrollment system
    public void enrollmentStudent(int studentID, int courseID) {
        // 1. Locate student and course in respective lists
        // 2. Verify not already enrolled
        // 3. Add bidirectional references
        // 4. Push operation to undo stack
    }

    // Additional features
    public void sortStudentsById(int courseID) { /* Sorts students in course */ }
    public void sortCourseById(int studentID) { /* Sorts courses for student */ }
}

```

```

    public void undo() { /* Reverses last operation */ }
    public void redo() { /* Reapplies undone operation */ }
}

```

4. Data Structures Implementation

4.1 Linked List Management

// Student list addition example

```

public void addStudent(int studentId) {
    if (studentHead == null) {
        studentHead = studentTail = new Student(studentId);
    } else {
        studentTail.next = new Student(studentId);
        studentTail = studentTail.next;
    }
}

```

// Course removal example

```

public void removeCourse(int courseId) {
    if ((courseHead == courseTail) && (courseHead.courseId == courseId)) {
        courseHead = courseTail = null; // Single element case
    } else if (courseHead.courseId == courseId) {
        courseHead = courseHead.next; // Head removal case
    } else {
        // Traverse list to find target course
        Course pred, tmp;
        for(pred=courseHead,tmp=courseHead.next;
            (tmp.courseId != courseId) && (tmp != null);
            tmp = tmp.next,pred = pred.next);

        if (tmp != null) {
            pred.next = tmp.next; // Bypass target node
            if (tmp == courseTail) {
                courseTail = pred; // Update tail if needed
            }
        }
    }
}

```

4.2 Stack Implementation

```

public class Stack {
    private Node head; // Top of stack

    public void push(Operation data) {
        head = new Node(data, head); // LIFO insertion
    }
}

```

```

    }

    public Operation pop() {
        Operation data = head.data; // Retrieve top
        head = head.next;          // Remove top
        return data;
    }

    private class Node {
        Operation data;
        Node next;
        // Constructors...
    }
}

class Operation {
    String operation; // "add" or "remove"
    Student student;
    Course course;
    // Constructor...
}

```

5. System Workflows

5.1 Enrollment Process

1. University.enrollmentStudent() called with (studentID, courseID)
2. System locates:
 - o Student in student linked list
 - o Course in course linked list
3. Verifies:
 - o Student not already in course
 - o Course not already in student's list
4. Performs:
 - o Course.addStudent(studentID)
 - o Student.addcourse(courseID)
5. Pushes operation to undo stack

5.2 Undo/Redo Mechanism

java
Copy

Download

```
public void undo() {
    Operation op = undo.pop();
    if (op.operation.equals("add")) {
        // Reverse an addition
        op.course.removeStudent(op.student.studentID);
        op.student.removeCourse(op.course.courseID);
        redo.push(new Operation("remove", op.student, op.course));
    } else {
        // Reverse a removal
        op.course.addStudent(op.student.studentID);
        op.student.addcourse(op.course.courseID);
        redo.push(new Operation("add", op.student, op.course));
    }
}
```

6. Design Advantages

1. Bidirectional Relationships
 - o Students know their courses and courses know their students
 - o Ensures data consistency
2. Efficient List Management
 - o Head/tail pointers enable $O(1)$ append operations
 - o Counter variables prevent full traversals for size checks
3. Clean Operation Tracking
 - o Stack-based undo/redo provides transactional safety
 - o Operation objects encapsulate all necessary context
4. Modular Sorting
 - o Sorting implemented at both Student and Course levels
 - o Consistent bubble sort implementation